

STAT 240

Week 10: More Text Data

Stat 240

Today

- Recap: Clustering
- More analysis of Text Data
 - Strings and StringR
 - Regular expressions

Brief Review of Clustering

Clustering

- Unsupervised learning method - No response Y to predict or use to evaluate our predictions
- Want to find groups of similar observations using numeric variables X
- Exploratory tool

K-means clustering

- Iterative procedure to find clusters in data
- Choose the number of clusters, k in advance
- Randomly assign each point to one of these clusters
- Run iterative procedure until stopping condition met

K-means Procedure

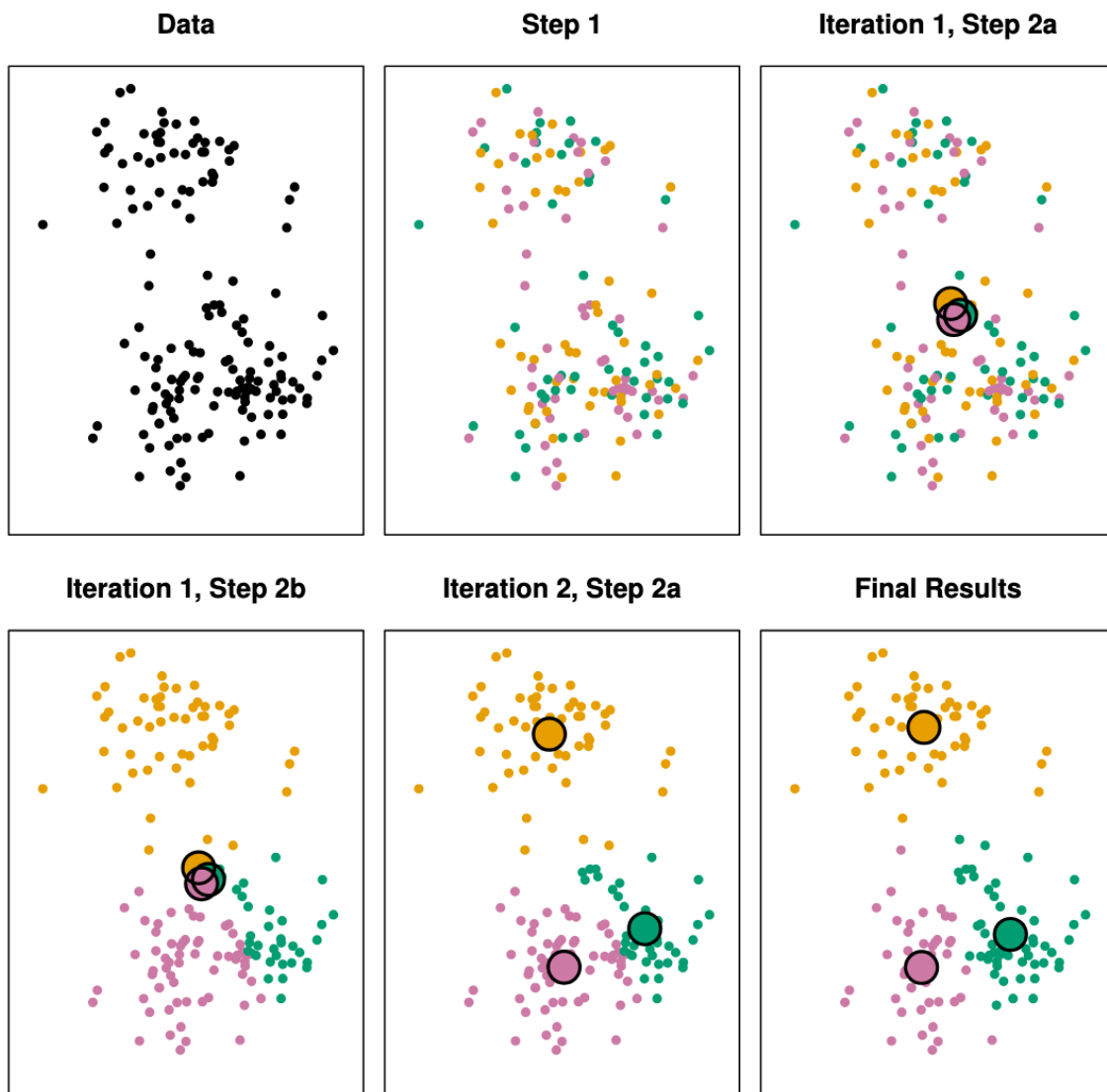
Repeatedly:

- Compute the centroid of points in each cluster
- Move points to clusters based on which centroid is closest (Euclidean distance)

Things to remember

- If cluster empty randomly make one data point the centroid
- How you place data in clusters initially will impact the clusters you get
- Run several clusters, pick the best one

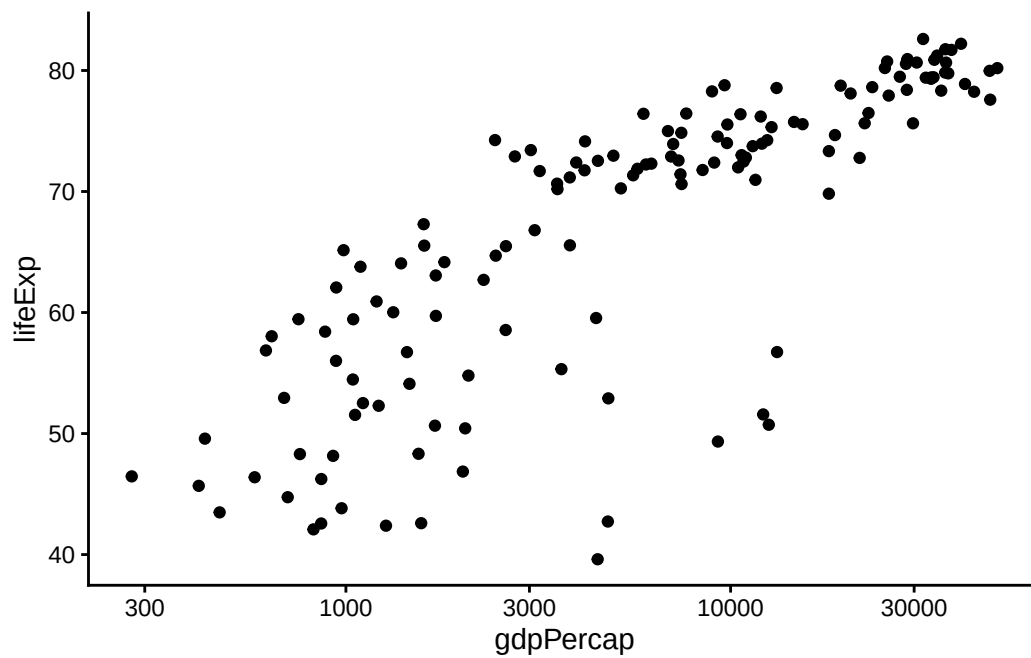
One Slide Summary - Taken from ISLR (book for 452)



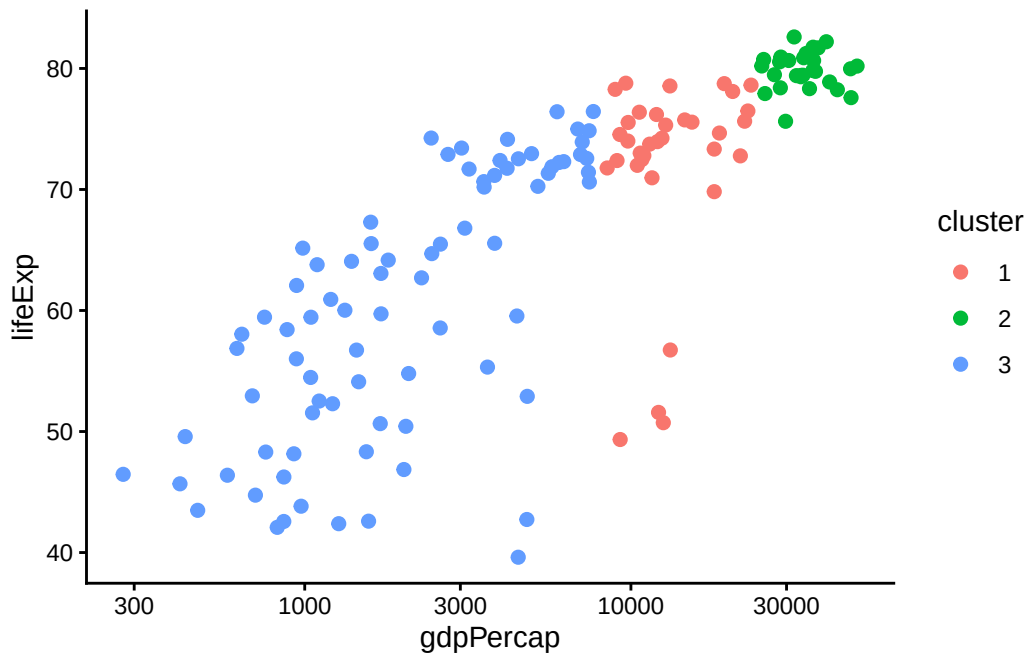
Another clustering example

```
ggplot(df, aes(x = gdpPercap, y = lifeExp)) +  
  geom_point() +
```

```
scale_x_log10() +  
theme_classic()
```



```
set.seed(123)  
k <- 3  
km <- kmeans(df |> select(gdpPercap, lifeExp), centers = k, nstart = 20)  
df$cluster <- as.factor(km$cluster)
```



Which countries are which

```
clust_1 <- df[df$cluster == 1, "country"]
clust_1$country[1:10]
```

```
[1] Argentina      Botswana        Brazil          Bulgaria
[5] Chile          Costa Rica      Croatia         Cuba
[9] Czech Republic Equatorial Guinea
142 Levels: Afghanistan Albania Algeria Angola Argentina Australia ... Zimbabwe
```

```
clust_2 <- df[df$cluster == 2, "country"]
clust_2$country[1:10]
```

```
[1] Australia Austria Bahrain Belgium Canada Denmark Finland
[8] France Germany Greece
142 Levels: Afghanistan Albania Algeria Angola Argentina Australia ... Zimbabwe
```

```
clust_3 <- df[df$cluster == 3, "country"]
clust_3$country[1:10]
```

```
[1] Afghanistan      Albania            Algeria
[4] Angola            Bangladesh        Benin
[7] Bolivia           Bosnia and Herzegovina Burkina Faso
[10] Burundi
142 Levels: Afghanistan Albania Algeria Angola Argentina Australia ... Zimbabwe
```

- K-means clustering seems to give reasonable results here
- Other issues which you will learn about in future classes (standardizing the data, etc)

Strings

Why we need strings

- Process and clean messy data, such as

```
prices <- c("$12.50", "$7.00", "$105.99")
```

- What about if we wanted to get the sum of these prices?

```
char_prices <- str_extract(prices, "\\d+\\.\\d+")
as.numeric(char_prices)
```

```
[1] 12.50  7.00 105.99
```

- Knowing some basic regular expressions very useful for things like this

The stringr package

```
library(stringr)
```

- The cleanest way to work with strings, previous methods were developed intermittently, harder to use
- The [corresponding chapter](#) in R4DS is an excellent reference
- We will only cover a small selection of tools today
- First see some key baseR functions

Some key functions

- The next few functions we will see are not specific to R, but rather R implementations of unix command line tools.
- These are useful to know about, if ever working with computing clusters, etc

grep

input: a vector or list

output: indices of elements with the pattern

```
ExampleText <- c("1 - small thing to do",  
                "2 - 2 big things that we're doing",  
                "Some Small things that were done 4 free - 333",  
                "four Things that we've done")  
grep("that", ExampleText)
```

```
[1] 2 3 4
```

```
grep("small", ExampleText) #which elements have small (but not Small)
```

```
[1] 1
```

```
grep("small", ExampleText, ignore.case = TRUE) #which elements have small or Small
```

```
[1] 1 3
```

```
grep("small", ExampleText, ignore.case = TRUE, value = TRUE)
```

```
[1] "1 - small thing to do"
```

```
[2] "Some Small things that were done 4 free - 333"
```

```
#which elements contain small, or Small, or smAll,...
```

```
Example1 <- "1 - small thing to do"
```

```
grep("mall", Example1)# element in which it exists (there is only one element)
```

```
[1] 1
```

- grep just says if “mall” exists (1) or not (0) in Example1

```
grep("mdall", Example1)
```

```
integer(0)
```

more on grep

- grep is not only an R function, can use it on the command line directly also
- I used this last week, to find a function written somewhere in one of many files

```
grep("my_function", list.files("some_folder", full.names = TRUE), value = TRUE)  
## Will return the files that contain "my_function"
```

substr

subset a string of text

input: a string of text

output: the elements from start to stop.

```
## Extract the letters between positions start and stop (inclusive)  
substr(Example1, start = 6, stop = 15)
```

```
[1] "mall thing"
```

gregexpr

input: a vector or list of text

output: list showing text start position and pattern length


```
ExampleText <- c("1 - small thing to do", "2 - 2 big things that we're doing",  
                "Some Small things that were done 4 free - 333",  
                "four Things that we've done")  
  
gregexpr("thing", ExampleText)
```

```
[[1]]  
[1] 11  
attr(,"match.length")  
[1] 5  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[2]]  
[1] 11  
attr(,"match.length")  
[1] 5  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[3]]  
[1] 12  
attr(,"match.length")  
[1] 5  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[4]]  
[1] -1  
attr(,"match.length")  
[1] -1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
gregexpr("small", ExampleText)
```

```
[[1]]  
[1] 5  
attr(,"match.length")  
[1] 5  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[2]]  
[1] -1  
attr(,"match.length")  
[1] -1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[3]]  
[1] -1  
attr(,"match.length")  
[1] -1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[4]]  
[1] -1  
attr(,"match.length")  
[1] -1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
gregexpr("small", ExampleText, ignore.case = TRUE)
```

```
[[1]]  
[1] 5  
attr(,"match.length")  
[1] 5  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[2]]  
[1] -1  
attr(,"match.length")  
[1] -1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[3]]  
[1] 6  
attr(,"match.length")  
[1] 5  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

```
[[4]]  
[1] -1  
attr(,"match.length")  
[1] -1  
attr(,"index.type")  
[1] "chars"  
attr(,"useBytes")  
[1] TRUE
```

gsub

replace a pattern within a string

input: a list, vector, or string

```
ExampleText <- c("1 - small thing to do",
                 "2 - 2 big things that we're doing",
                 "Some Small things that were done 4 free - 333",
                 "four Things that we've done")

gsub("thing", "stuff", ExampleText) # replace "thing" with "stuff"
```

```
[1] "1 - small stuff to do"
[2] "2 - 2 big stuffs that we're doing"
[3] "Some Small stuffs that were done 4 free - 333"
[4] "four Things that we've done"
```

```
gsub("thing", "stuff", ExampleText, ignore.case = TRUE) # replace "thing", "Thing", "ThInG",
```

```
[1] "1 - small stuff to do"
[2] "2 - 2 big stuffs that we're doing"
[3] "Some Small stuffs that were done 4 free - 333"
[4] "four stuffs that we've done"
```

Back to stringr

- Four key functions from **stringr**
 - Match a string and extract it: **str_extract** / **str_extract_all**
 - Detect a matching string **str_detect**
 - Replace one string with another **str_replace** / **str_replace_all**
 - Split a string on a substring **str_split**

Key stringr Functions

- These functions work with both fixed/literal strings like **fish** as well as regular expressions like **f[aeiou]sh**
- We'll see a bit more about regular expressions now
- These are always awkward to work with, good to know the basics but google is your friend

Literal vs metacharacters

- Letters and numbers are **literal characters** - they match exactly

```
fruit[1:5]
```

```
[1] "apple"      "apricot"    "avocado"    "banana"     "bell pepper"
```

```
# Find all the fruits from fruit that contain "berry"
fruit[str_detect(string = fruit, pattern = "berry")]
```

```
[1] "bilberry"    "blackberry" "blueberry"   "boysenberry" "cloudberry"
[6] "cranberry"   "elderberry" "goji berry"  "gooseberry"  "huckleberry"
[11] "mulberry"    "raspberry"  "salal berry" "strawberry"
```

-
- Most punctuation characters like ., +, [], ? are **metacharacters** that have special behaviour

```
# Find all the fruits that contain an "a",
# followed by any three characters, followed by an "e"
fruit[str_detect(string = fruit, pattern = "a...e")]
```

```
[1] "apple"      "blackberry" "mandarine"   "nectarine"   "pineapple"
[6] "pomegranate" "raspberry"  "salal berry"
```

Escaping

What if you want to match a literal .? Need an escape, which tells the regexp to literally match metacharacters

- Like strings, regexps use the backslash, \, for escaping
- Since regexps are also strings, this makes things a bit tricky!
- E.g., to create the regular expression \., use the string \\..

-
- E.g., to create the regular expression \., use the string \\..

```
mystrings <- c("abc", "a.c")
str_extract(mystrings, ".")
```

```
[1] "a" "a"
```

```
str_extract(mystrings, "\\.")
```

```
[1] NA  "."
```

Anchors

- Regexps match any part of a string. If you want to match at the start or end, use an anchor
 - Use `^` to match the start
 - Use `$` to match the end

```
fruit[str_detect(fruit, "^a")]
```

```
[1] "apple"  "apricot" "avocado"
```

```
fruit[str_detect(fruit, "a$")]
```

```
[1] "banana"  "cherimoya" "feijoa"  "guava"    "papaya"   "satsuma"
```

More Anchors

- To force a match to only a full string, use both anchors. Notice the difference:

```
fruit[str_detect(fruit, "apple")]
```

```
[1] "apple"  "pineapple"
```

```
fruit[str_detect(fruit, "^apple$")]
```

```
[1] "apple"
```

- What about fruits which are **exactly** 5 letters long?

```
fruit[str_detect(fruit, "^.....$")]
```

```
[1] "apple" "grape" "guava" "lemon" "mango" "olive" "peach"
```

Character classes

- A character class or character set allows you to match any character in a set, and you can construct your own sets with `[ ]` and (if needed) special characters
- `[abc]` matches "a", "b", "c"

```
# Fruits that start with "a", "b" or "c"  
fruit[str_detect(fruit, "^[abc]")] [1:8] #print 1st 8 only
```

```
[1] "apple"      "apricot"      "avocado"      "banana"      "bell pepper"  
[6] "bilberry"   "blackberry"   "blackcurrant"
```

- `[^abc]` matches everything except "a", "b", "c"

```
# Fruits that start with a letter other than "a", "b" or "c"  
fruit[str_detect(fruit, "^[^abc]")] [1:8] # 1st 8 only
```

```
[1] "damson"      "date"          "dragonfruit"  "durian"      "eggplant"  
[6] "elderberry"  "feijoa"        "fig"
```

- `-` indicates a range, e.g., `[a-z]` matches any lowercase letter and `[0-9]` matches any digit

```
fruit[str_detect(fruit, "[w-z]")] [1:8]
```

```
[1] "bilberry"      "blackberry"    "blueberry"    "boysenberry"  "canary melon"  
[6] "cherimoya"     "cherry"        "cloudberry"
```

- `\` escapes special characters. `[\^-\]` matches `^`, `-`, or `]`

Useful shortcuts

- `.` matches any character (except newline)
- `\d` matches any digit
- `\D` matches anything that isn't a digit
- `\s` matches any whitespace (space, tab, newline, etc)
- `\S` matches anything that isn't whitespace
- `\w` matches any "word" character (letters and numbers)
- `\W` matches any "non-word" character

Adding `*` and `+` quantifiers to `.`

- The combinations `.*` and `.+` match multiple characters.
 - E.G., `f.*n` matches `f` followed by 0 or more characters, followed by `n`.
 - `f.+n` matches `f` followed by 1 or more characters, followed by `n`.

```
mystrings <- c("fun", "flin flon", "flin flam", "fn")
pattern1 <- "f.*n"
pattern2 <- "f.+n"
str_extract(mystrings, pattern1)
```

```
[1] "fun"      "flin flon" "flin"      "fn"
```

```
str_extract(mystrings, pattern2)
```

```
[1] "fun"      "flin flon" "flin"      NA
```

"Greedy" matching with `*`

- The `*` quantifier matches the longest possible string.

```
mystrings <- c("fun", "fun, fun, fun", "fn")
pattern1 <- "f.*n"
str_extract(mystrings, pattern1)
```

```
[1] "fun"      "fun, fun, fun" "fn"
```


Numerical quantifiers

- Use `{n}` to require exactly `n` matches, `{n,}` to require `n` or more, `{,m}` at most `m`, and `{n,m}` between `n` and `m`.

```
str_extract(mystrings, "f.{6}n")
```

```
[1] NA          "fun, fun" NA
```

```
str_extract(mystrings, "f.{1,13}n")
```

```
[1] "fun"          "fun, fun, fun" NA
```

Exercise

- What about fruits which are **exactly** 5 letters long?

```
fruit[str_detect(fruit, "^.....$")]
```

```
[1] "apple" "grape" "guava" "lemon" "mango" "olive" "peach"
```

- Can we do this more easily now? (maybe we type too many ...)

Solutions

- Which one of these is better/safer?

More Solution

- You probably want the second solution here, maybe not the first
- Important to think about edge cases like these and check them

Any character

Consider the regexp `p.n`

```
mystrings <- c("pineapple", "apple", "pen")
str_detect(mystrings, "p.n")
```

```
[1] TRUE FALSE TRUE
```

```
str_extract(mystrings, "p.n")
```

```
[1] "pin" NA "pen"
```

```
str_replace(mystrings, "p.n", "PPP")
```

```
[1] "PPPeapple" "apple" "PPP"
```

- Note that the replacement string in `str_replace` is literal:

```
str_replace(mystrings, "p.n", "p.n")
```

```
[1] "p.neapple" "apple" "p.n"
```

Digits

```
mystrings <- c("$55.30", "$22.43")
str_extract(mystrings, "\\d") #Note that we need to "escape"
```

```
[1] "5" "2"
```

```
str_extract_all(mystrings, "\\d")
```

```
[[1]]
```

```
[1] "5" "5" "3" "0"
```

```
[[2]]
```

```
[1] "2" "2" "4" "3"
```

Whitespace

```
mystrings <- c("it was the best of times",
              "it was the worst of times")
str_detect(mystrings, "\\s") #Note that we need to "escape"
```

```
[1] TRUE TRUE
```

```
str_extract(mystrings, "\\s")
```

```
[1] " " " " " "
```

```
str_extract(mystrings, "\\S")
```

```
[1] "i" "i"
```

Words and non words

```
mystrings <- c("My lunch cost", "$10.00")
str_extract_all(mystrings, "\\w") #Note that we need to "escape"
```

```
[[1]]
[1] "M" "y" "l" "u" "n" "c" "h" "c" "o" "s" "t"
```

```
[[2]]
[1] "1" "0" "0" "0"
```

```
str_extract_all(mystrings, "\\W")
```

```
[[1]]
[1] " " " " " "
```

```
[[2]]
[1] "$" "."
```

Exercise

Last week we saw how many lines in the novel Emma contained the name Emma.

- Use the tools we have seen in this class to determine what proportion of lines in this text contain a word that includes the two letters **em**, followed by any other letter, followed by **a**. Are any of these not just **emma**? You should include both upper and lower case words!

```
library(janeaustenr)
data("emma")
emma_lower <- str_to_lower(emma)
emma_lower[1:4]
```

```
[1] "emma"          ""              "by jane austen" ""
```

Solution

What are the differences?

- Could we get the right answer with this patter?

Modifying this pattern?

Operators

- Using **|** in a regular expression is like the logical OR

```
mystrings <- c("one fish", "two fish", "red fish", "blue fish")
str_detect(mystrings, "red|blue")
```

```
[1] FALSE FALSE  TRUE  TRUE
```

- Often use **|** with parentheses

```
mystrings <- c("theater", "theatre")
str_extract(mystrings, "theat(er|re)")
```

```
[1] "theater" "theatre"
```

Operator Precedence

- `ab+` is equivalent to `a(b+)` not `(ab)+`

```
mystrings <- c("baby", "babble", "bababa")
str_extract(mystrings, "ab+")
```

```
[1] "ab" "abb" "ab"
```

- `^a|b` is equivalent to `(^a)|(b$)`

```
mystrings <- c(mystrings, "ability", "nab")
str_detect(mystrings, "^a|b$")
```

```
[1] FALSE FALSE FALSE TRUE TRUE
```

- When in doubt use parentheses!

Grouping and capturing

- Parentheses also create **capturing groups** that allow you to use sub-components of a match
- For example, you can refer back to a capturing group with a back reference: `\1` refers to the match contained in the first parenthesis, `\2` in the second parenthesis, etc.

-
- Suppose you want to find fruits with a repeated pair of letters:

```
fruit[str_detect(fruit, "(..)\1")] [1:4] #1st 4 only for space
```

```
[1] "banana" "coconut" "cucumber" "jujube"
```

- Or find words that start and end with the same pair of letters:

```
words[str_detect(words, "^(..).*\1$")] [1:4] #1st 4 only for space
```

```
[1] "church" "decide" "photograph" "require"
```

Exercise

- Use a regular expression to find any fruits where either:
 - The first two letters are repeated directly (e.g., coconut)
 - The fruit name contains two words

Solution

Final Comment on Strings and Regular Expressions

Some people, when confronted with a problem, think ‘I know, I’ll use regular expressions.’ Now they have two problems.

–Jamie Zawinski (Emacs forum)